VINUNIVERSITY

Lab Practice № 7:
Stack

COMP1020: OOP, Algorithms, and Data Structures **Week 08**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your classes and methods should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables, methods and classes should have meaningful names, and should be followed Java programming convention.
- Academic honesty is required in all work you submit to be graded. Although you can discuss among yourself at high level how to solve the lab problem, you should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions.
- You should upload your .java file(s) to both the Canvas and CMS Autograder before the end of the laboratory session unless the instructor gave a specified deadline.
  (a) No Canvas submission will get 0 point (of course).
  (b) No CMS Autograder submission will get 20% deduction.
- Submit separate *.java file for each Lab problem with the exact file name specified in each lab question (do not add your student ID or your name). For example: **HelloWorld.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
  (a) No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
  (b) The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
  (c) The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

CMS Autograder

Direct access to this CMS lab is followed: https://www.cmsvinuni.online
Note: Due to the current limitation of the Autograder, you must comment out all the `package` declaration before submission.

Problem: Stack Stutter

Stack is a collection based on the principle of adding elements and retrieving them in the opposite order. This is called Last-In, First-Out ("LIFO").

Method	Description
`push(E item)`	Pushes an item onto the top of the stack.
`pop()`	Removes and returns the item at the top of the stack.
`peek()`	Returns the item at the top of the stack without removing it.
`isEmpty()`	Returns `true` if the stack is empty, else `false`.
`search(Object o)`	Returns the 1-based position from the top of the stack if found, else `-1`.
`size()`	Returns the number of elements in the stack.
`clear()`	Removes all elements from the stack.
`contains(Object o)`	Returns `true` if the stack contains the specified element.

Your task:
Write a program and name it `StackStutter.java` Given a stack of integers, duplicate every element in the stack so that each value appears twice in succession, while preserving the original order of elements. Using 'Stack' is mandatory.
Input
The input contains a block of lines (terminated by a line containing `#`). Each line contains a single integer to be pushed onto the stack. The first number entered will be at the bottom, and the last number entered will be at the top.
Output
A single line containing the stack after performing the stutter operation. The output should list the elements from bottom to top, separated by spaces.
Example See Figure 1
Logic You Can Follow

- Reverse the original stack: Since stacks are LIFO (last-in, first-out), you can reverse the order of elements by popping them into another temporary stack.
- Duplicate each element: Now that the original elements are in reverse order in the temporary stack, pop each item, and push it twice into the original stack.
- Restore original order: At this point, the original stack contains duplicated items, but the order is reversed again. So, reverse it one more time using the temporary stack.

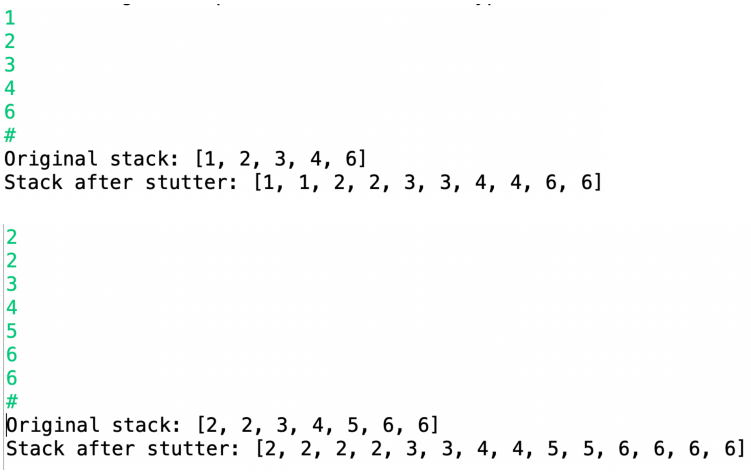

Figure 1: Example testcase of the Problem